Day 132: The Week 19 Build — a Self-Supervised Pretext Task, and Every Representation of the Week

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 7 | Complete study guide

Watch: Computer Vision - Lecture 11.3 (Self-Supervised Learning: Pretext Tasks) (Tübingen Machine Learning) | Self-Supervised Learning: Self-Prediction and Contrastive Learning, NeurIPS 2021 tutorial

Script: `day132_self_supervised_build.py` · every number in this guide is copied by program from `day132_run_output.txt`, the real output of that script.

A network learns to say whether a digit was turned by 0, 90, 180 or 270 degrees, with no digit labels. It gets the turn right 0.9927 of the time, yet its 128 numbers are no better for recognising digits than the same network's untrained starting weights (0.5488 against 0.5382 with 10 labels per digit). The week's six parts, copied in verbatim, give five ways to learn without labels, all judged the same way. Contrastive learning wins with few labels (0.8157, raw pixels 0.6078); the VAE's means win for neighbours and clusters; the rotation network is the best detector of upside-down digits (0.9951). Which representation is best depends on what it is for.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

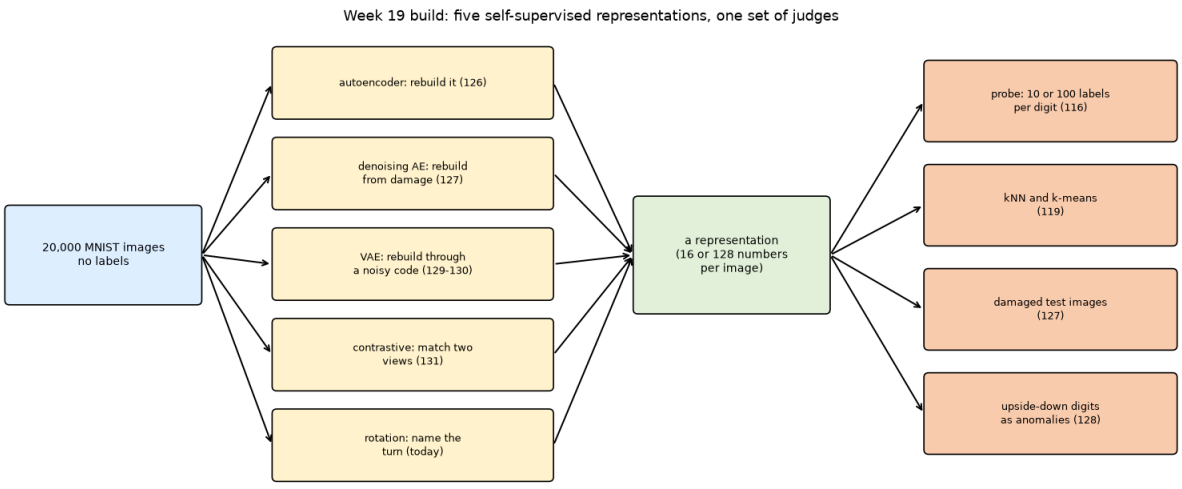
A **pretext task** is a made-up task whose answers come free with the data: here, turn each image by a random multiple of 90 degrees and ask which. Solving it is not the goal; the hope is that the network's inner layers learn something useful on the way, which a **downstream task** (recognising digits with few labels) can reuse.

Day 132 is the week's **build**: the exercise parts of Days 126 to 131 are copied in unchanged, and every representation of the week is judged by the same tests.

REAL-WORLD SCENARIO

Training new shop staff by having them sort returned stock: the sorting itself is not the job, but it should teach them the products. Whether it does depends on the sorting rule: sorting by box colour teaches nothing about what is inside.

SYSTEM DIAGRAM



Five ways to learn a representation without labels, and the same judges for each.

Data: MNIST (cached by Day 126). Every representation is learned from the same 20,000 unlabelled training images; images 60,000–69,999 are the test set, never trained on. Labels are used only to judge. Day 116's softmax regression, Day 119's k-means and ARI, Day 117's ROC AUC and Day 123's PCA are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Checks: the parts are verbatim, and the new pieces are right

DEFINITION — start here, no prior knowledge assumed

The script parses the six exercise files and finds every function in itself, character for character. The new pieces (turning images, the 4-way cross-entropy) are checked by hand and against finite differences.

REAL-WORLD SCENARIO

Checking that every part on the assembly line is the part that passed inspection, before switching the line on.

MATHEMATICS

$$\text{cross-entropy} = -\log p_{\text{answer}}, \quad p = \text{softmax}(\text{scores}) \quad \partial/\partial\text{scores} = (p - \text{one-hot answer}) / n$$

IMPLEMENTATION

```
def rotate4(X):
    I = X.reshape(-1, 28, 28)
    return np.vstack([np.rot90(I, k, axes=(1, 2)).reshape(len(X), -1) for k in range(4)]),
        np.repeat(np.arange(4), len(X))
def softmax_ce(L, y):
    m = L.max(1, keepdims=True); P = np.exp(L - m); P /= P.sum(1, keepdims=True); n = len(y)
    return float(-np.log(P[np.arange(n), y] + 1e-30).mean()), (P - np.eye(L.shape[1],
        dtype=L.dtype)[y]) / n
```

ACTUAL OUTPUT

```
functions and classes from day126_exercise.py to day131_exercise.py found here, character for
    character: 34 of 34
rotate4: labels [0, 1, 2, 3]; turning the 270-degree copy a further 90 degrees gives the original
    back: True
softmax_ce for scores (2, 0, 0, 0), answer 0: 0.3408; by hand -log(e^2 / (e^2 + 3)) = 0.3408
its gradient against finite differences: largest gap 4.0e-10
Actual output from this lesson's run.
```

What it means

All 34 of 34 parts are present. Four quarter turns bring an image back to itself; the loss for scores (2, 0, 0, 0) is 0.3408, as by hand; its gradient matches finite differences to 4.0e-10.

WHY CHOOSE THIS?

The build cannot drift from the parts it claims to use.

WHY NOT THIS? (TRADEOFFS)

A verbatim check proves the copy, not that the parts suit today's task.

2. The pretext task: which way was this digit turned?

DEFINITION — start here, no prior knowledge assumed

Every training image appears four times, turned by 0, 90, 180 and 270 degrees; the label is the turn. A network 784 → 256 → 128 → 4 (Day 126's parts and Adam) learns to name it. Its 128-number layer is the representation.

REAL-WORLD SCENARIO

Being handed photos from an album, some upside down or sideways, and putting each the right way up.

SPECIFICATION

Data	20,000 images x 4 turns = 80,000 per epoch, no digit labels
Training	10 epochs, batches of 128, Adam at 0.001
Test	10,000 test digits x 4 turns

IMPLEMENTATION

```
def train_rotation(X, sizes, epochs=10, batch=128, lr=1e-3, seed=0):
    acts = ["relu"] * (len(sizes) - 2) + ["linear"]; net = init_net(sizes, seed); opt = Adam(net,
        lr)
    R, r = rotate4(X); rng = np.random.RandomState(seed + 1); hist = []
    for _ in range(epochs):
        o = rng.permutation(len(R)); losses = []
        for s in range(0, len(R), batch):
            A = forward(net, R[o[s:s + batch]], acts); loss, d = softmax_ce(A[-1], r[o[s:s +
                batch]])
            opt.step(net, backward(net, A, acts, d)[0]); losses.append(loss)
        hist.append(float(np.mean(losses)))
    return net, hist
```

ACTUAL OUTPUT

```
network 784 -> 256 -> 128 -> 4, trained on 20,000 images x 4 turns, 10 epochs (35 s); no digit labels used
training loss by epoch: 0.102, 0.030, 0.024, 0.017, 0.017, 0.012, 0.012, 0.010, 0.011, 0.009 (guessing among 4
    turns: 1.386)
10,000 test digits x 4 turns: the turn named correctly 0.9927 of the time
   digit  correct  most common mistake
       0  0.9737    90 called 180 (0.006 of this digit's turned copies)
       1  0.9976   270 called  90 (0.002 of this digit's turned copies)
       2  0.9910    90 called   0 (0.002 of this digit's turned copies)
       3  0.9990     0 called 180 (0.000 of this digit's turned copies)
       4  0.9959     0 called  90 (0.001 of this digit's turned copies)
       5  0.9922   270 called   0 (0.001 of this digit's turned copies)
       6  0.9893     0 called 180 (0.002 of this digit's turned copies)
       7  0.9925   270 called   0 (0.003 of this digit's turned copies)
       8  0.9954   270 called 180 (0.001 of this digit's turned copies)
       9  0.9995   180 called 270 (0.000 of this digit's turned copies)
hardest digits: 0 (0.974), 6 (0.989), 2 (0.991)
a 6 turned 180 degrees looks like an upright 9: named 'no turn' 0.006 of the time; a 9 turned 180, 0.000
Actual output from this lesson's run.
```

What it means

The task is easy for this network: the loss falls from 0.102 to 0.009 (guessing would be 1.386), and the turn is right 0.9927 of the time on test digits. A prediction made before the run was wrong: 0, 1 and 8 look nearly the same after a half turn, so they were expected to be the hardest. Only 0 is noticeably harder (0.9737); 1 and 8 score 0.9976 and 0.9954, so handwriting keeps enough slant and stroke detail to tell which way up they are. A 6 turned upside down is named 'no turn' only 0.006 of the time, although it resembles a 9.

WHY CHOOSE THIS?

Labels for free, and a task any image can supply.

WHY NOT THIS? (TRADEOFFS)

An easy task can be solved by cues that have little to do with what the digit is (section 3).

3. Five self-supervised representations, judged with few labels

DEFINITION — start here, no prior knowledge assumed

Each representation is judged four ways: a **linear probe** (Day 116's softmax regression) with 10 or 100 labelled images per digit, a 3-nearest-neighbour classifier with 5,000 labelled images, and k-means with 10 clusters, scored by ARI against the true digits. Baselines: raw pixels, PCA, and an **untrained network** of the rotation network's shape (its random starting weights).

REAL-WORLD SCENARIO

Five applicants who trained in different ways, given the same short test with the same handful of worked examples.

SPECIFICATION

Representations	autoencoder 128 (Day 126), denoising autoencoder 128 (Day 127, half the pixels dropped), VAE means 16 (Day 130), contrastive 128 (Day 131's settings), rotation 128 (section 2)
Extra rows	the rotation network's first layer (256 numbers); a rotation network stopped after 1 epoch; the untrained network
Probe	standardised features, softmax regression with $\lambda = 0.01$ and 500 steps; 10 per digit averaged over 3 draws

IMPLEMENTATION

```
def probe_fit(R, y, n_per, seed=0):
    rng = np.random.RandomState(seed)
    tr = np.concatenate([rng.choice(np.where(y[:60000] == c)[0], n_per, replace=False) for c in
        range(10)])
    mu, sd = R[tr].mean(0), R[tr].std(0) + 1e-6
    W, b = d116.fit_softmax(((R[tr] - mu) / sd).astype(np.float64), y[tr], 10, lam=1e-2,
        steps=500)
    return lambda Z: (((Z - mu) / sd) @ W + b).argmax(1)
def knn_acc(R, y, tr, te, k=3):
    nn = np.argsort(d119.sqdist(R[te].astype(np.float64), R[tr].astype(np.float64)), 1)[:, :k]
    return float(np.mean(np.array([np.bincount(r, minlength=10).argmax() for r in y[tr][nn]]) ==
        y[te]))
```

ACTUAL OUTPUT

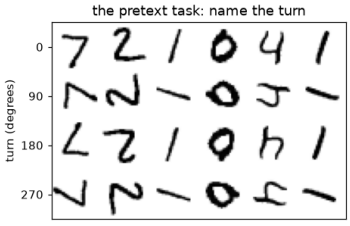
```
   representation        kNN     probe 10/digit (3 draws)  probe 100  k-means ARI  train s
   raw pixels 784        0.9085  0.6078 (0.587-0.629)      0.8059      0.3867      -
   PCA 16                0.9095  0.7029 (0.671-0.728)      0.8297      0.3870      -
   PCA 128               0.9155  0.5356 (0.532-0.538)      0.8217      0.3975      -
   autoencoder 128       0.9110  0.7100 (0.692-0.744)      0.8824      0.4039      47
   denoising AE 128      0.9190  0.7693 (0.747-0.783)      0.8779      0.4112      49
   VAE means 16          0.9205  0.7520 (0.711-0.785)      0.8575      0.4331      68
   contrastive 128       0.9135  0.8157 (0.765-0.853)      0.9311      0.3773      114
   rotation 128          0.7065  0.5488 (0.509-0.594)      0.7754      0.0407      sec. 2
   rotation, layer 1     0.8760  0.6026 (0.598-0.610)      0.8664      0.3472      sec. 2
   rotation, 1 epoch     0.7380  0.6308 (0.606-0.669)      0.7887      0.0720      3
   untrained net 128     0.8000  0.5382 (0.463-0.590)      0.7714      0.1976      -
best by 10 labels per digit: contrastive 128 (0.8157); raw pixels 0.6078
best by 100 labels per digit: contrastive 128 (0.9311); raw pixels 0.8059
best by kNN: VAE means 16 (0.9205); raw pixels 0.9085
best by k-means ARI: VAE means 16 (0.4331); raw pixels 0.3867
Actual output from this lesson's run.
```

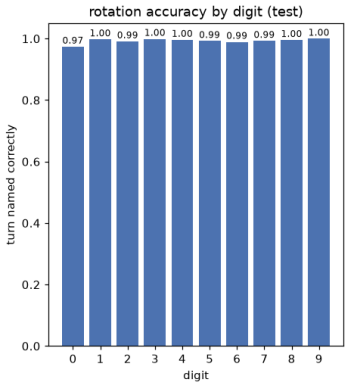
What it means

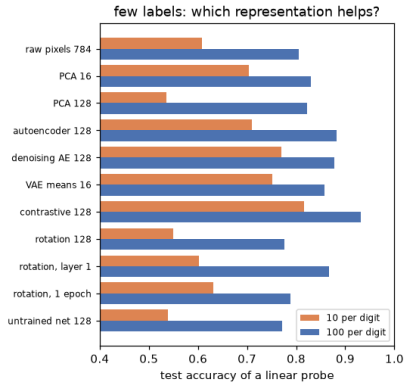
With few labels, contrastive learning is best by a wide margin: 0.8157 with 10 per digit and 0.9311 with 100, against raw pixels' 0.6078 and 0.8059. The denoising autoencoder (0.7693) and the VAE means (0.7520) come next. The VAE means are best for neighbours and clusters (kNN 0.9205, ARI 0.4331), although they hold only 16 numbers.

What it means

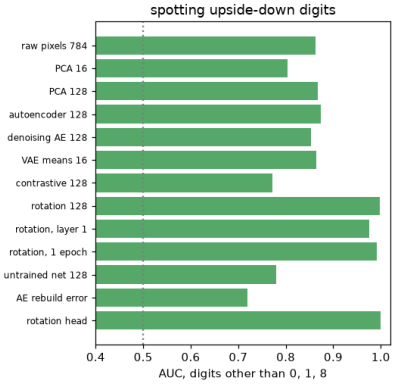
The rotation network's 128 numbers are the worst: 0.5488 with 10 per digit, below raw pixels, and almost exactly what its untrained starting weights give (0.5382); their clusters barely match the digits (ARI 0.0407). Two measurements say where the knowledge went. The first layer is better (0.6026, and 0.8664 with 100 per digit), so more of what helps with digits survives in the first layer than in the last hidden one. And training longer made that layer worse for digits: 0.6308 after 1 epoch, 0.5488 after 10, while the pretext accuracy rose. A likely reason, not tested here: telling up from down on centred digits needs only coarse cues such as where the ink sits, and a network that has found them has no reason to learn more.

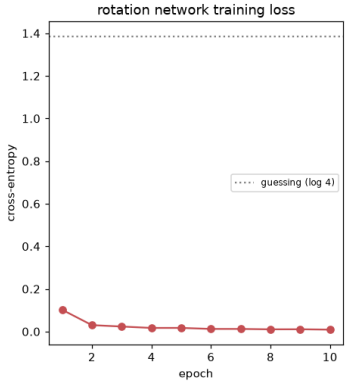












Top: six test digits in four turns; rotation accuracy per digit; probe accuracy for every representation. Bottom: probe accuracy on damaged test images; AUC for spotting upside-down digits; the rotation network's training loss.

WHY CHOOSE THIS?

One table that compares the whole week on equal terms.

WHY NOT THIS? (TRADEOFFS)

Every method had one setting and one training run; the 10-label probes vary by up to 0.13 between draws (the ranges in brackets).

4. Robustness: the same probes, tested on damaged images

DEFINITION — start here, no prior knowledge assumed

The 100-per-digit probes, trained on clean images, are tested on images damaged with Day 127's tools: Gaussian noise (0.3), half the pixels dropped, or a 10 x 10 square hidden.

REAL-WORLD SCENARIO

A cashier trained on crisp banknotes, handed crumpled and torn ones.

ACTUAL OUTPUT

```
   representation                     clean            noise 0.3     half pixels gone  10x10 square hidden
   raw pixels 784                  0.8059               0.1396               0.6973               0.5318
   PCA 16                          0.8297               0.8034               0.6557               0.5237
   PCA 128                         0.8217               0.6918               0.5862               0.4524
   autoencoder 128                 0.8824               0.6888               0.6745               0.5137
   denoising AE 128                0.8779               0.5264               0.7780               0.6634
   VAE means 16                    0.8575               0.4625               0.7700               0.6144
   contrastive 128                 0.9311               0.2943               0.7991               0.6503
   rotation 128                    0.7754               0.4136               0.5089               0.2985
   rotation, layer 1               0.8664               0.5017               0.6742               0.4118
   rotation, 1 epoch               0.7887               0.5144               0.4531               0.3494
   untrained net 128               0.7714               0.5240               0.4457               0.4449
best under noise 0.3: PCA 16 (0.8034)
best under half pixels gone: contrastive 128 (0.7991)
best under 10x10 square hidden: denoising AE 128 (0.6634)
caution: the contrastive network trained on views with noise and erased patches (Day 131's augment), and the
denoising autoencoder on dropped pixels; the others never saw damage
Actual output from this lesson's run.
```

What it means

No representation wins everywhere. Strong noise breaks raw pixels (0.1396) and the contrastive representation (0.2943), even though Day 131's views included noise of 0.1; PCA with 16 numbers barely notices it (0.8034). With half the pixels gone the contrastive representation is best (0.7991); with a hidden square, the denoising autoencoder (0.6634). Both had seen similar damage in training (erased patches, dropped pixels), so these wins are partly built in. Under a hidden square the rotation representation is the weakest of all (0.2985).

WHY CHOOSE THIS?

Shows what each training method made the representation ignore.

WHY NOT THIS? (TRADEOFFS)

Robustness follows the damage seen in training; a new kind of damage can undo it.

5. Anomalies: which representation notices a digit turned upside down?

DEFINITION — start here, no prior knowledge assumed

2,000 upright test digits are normal; the same digits turned 180 degrees are the anomalies. Each image's score is its distance to the nearest of 5,000 upright training images in each representation (Day 128's nn_score); AUC 1 is perfect, 0.5 is guessing. The second column leaves out 0, 1 and 8, which were expected to be ambiguous upside down. The rotation network's own head gives a further score: one minus its probability of 'no turn'.

REAL-WORLD SCENARIO

A proof-reader who has spent weeks fixing upside-down pages spots one at once; a fast reader may not notice.

ACTUAL OUTPUT

```
normal: 2,000 upright test digits; anomalies: the same digits turned 180 degrees
score: distance to the nearest of 5,000 upright training images, in each representation (Day 128's
    nn_score)
   score                  AUC all digits  AUC without 0, 1, 8
   raw pixels 784         0.8019          0.8632
   PCA 16                 0.7619          0.8030
   PCA 128                0.8068          0.8680
   autoencoder 128        0.8191          0.8737
   denoising AE 128       0.8116          0.8524
   VAE means 16           0.8281          0.8637
   contrastive 128        0.7290          0.7713
   rotation 128           0.9951          0.9983
   rotation, layer 1      0.9405          0.9757
   rotation, 1 epoch      0.9809          0.9914
   untrained net 128      0.7353          0.7795
   AE rebuild error       0.6737          0.7198
   rotation head 1-P(0)   0.9999          0.9999
Actual output from this lesson's run.
```

What it means

Here the rotation network is far ahead: its 128 numbers give AUC 0.9951, its head 0.9999, against 0.7290 to 0.8281 for the representations not trained on turning, and 0.6737 for Day 128's reconstruction error. The representation that failed at digits in section 3 is the best at the thing it was trained on. A representation is good *for* something.

WHY CHOOSE THIS?

A pretext task that matches the anomaly gives a near-perfect detector.

WHY NOT THIS? (TRADEOFFS)

It detects only that anomaly; Day 128's scores aim at anything unusual.

Interview preparation — technical questions

Q: How do you evaluate a self-supervised representation?

A: Freeze it and train a linear probe with few labels; here 10 per digit separated the methods (0.5488 to 0.8157).

Q: Why compare against an untrained network?

A: A random network's features are a real baseline: the rotation representation scored 0.5488 against its starting weights' 0.5382.

Q: Which layer should you use?

A: Measure it: the rotation network's first layer gave 0.6026, its last hidden layer 0.5488.

Q: How do you check a week's parts were reused unchanged?

A: Parse each exercise file and search for every function's text in the build: 34 of 34.

Q: What does high pretext accuracy guarantee?

A: Nothing downstream: 0.9927 on rotations, yet worse than raw pixels for digits.

Interview preparation — theoretical questions

Q: Why can a pretext task fail?

A: If it can be solved by shortcuts unrelated to the downstream task, the network learns the shortcut.

Q: Why did contrastive learning do best with few labels?

A: Its views were designed so that what stays the same is the digit's identity.

Q: Why might longer pretext training hurt?

A: Here the last hidden layer became less useful for digits as the turning task was learned better: 0.6308 after 1 epoch, 0.5488 after 10 (why was not tested).

Q: What decides which representation is best?

A: The downstream task: digits favoured contrastive and VAE codes, upside-down detection favoured rotation.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Self-supervised learning works only as well as its invented task matches the real one. Predicting rotations was easy, and the network learned to turn digits, not to recognise them; contrastive views, built to keep identity, gave the best few-label classifier of the week. Every number here was measured on the same images with the same judges, including the untrained baseline that exposed the rotation network.

- Rotation: 0.9927 of turns named; digits from its representation 0.5488 with 10 labels, untrained 0.5382.
- Few labels: contrastive 0.8157, denoising AE 0.7693, VAE means 0.7520, raw pixels 0.6078.
- Neighbours and clusters: VAE means best (kNN 0.9205, ARI 0.4331).
- Upside-down digits: rotation representation AUC 0.9951, its head 0.9999.

Suggested watch order, if starting from zero

- 1. Computer Vision - Lecture 11.3 (Self-Supervised Learning: Pretext Tasks) (Tübingen Machine Learning) — pretext tasks including rotation prediction, in a university lecture.
- 2. Self-Supervised Learning: Self-Prediction and Contrastive Learning, NeurIPS 2021 tutorial — self-prediction and contrastive learning surveyed.

Day 133 preview

- **Generative vs discriminative models**: learning how the data is made against learning only the boundary.